

Construindo seu Chat de IA Personalizado

Parte 3: Front-end com IA e Produção

Interface própria em React, API com FastAPI, PDF na base e deploy robusto no Render, com o mesmo portão de qualidade

Parte 3 de 3 · 08/10 · Apostila do aluno

Prof. Thiago Azeredo Rodrigues

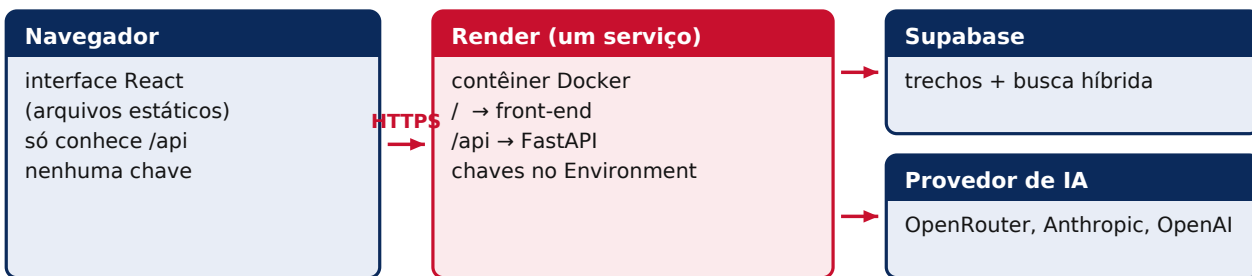
Parte 1 (24/09): CI/CD e Deploy

Parte 2 (01/10): RAG e Base de Conhecimento

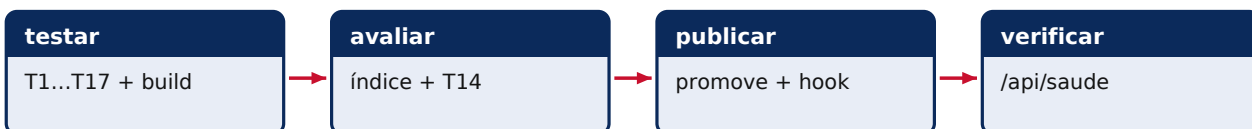
01 O que você vai construir hoje

Nas Partes 1 e 2, o assistente ganhou deploy automático e uma base de conhecimento. Ele funciona, mas tem cara de protótipo: a interface é a padrão do Gradio e ele mora num Space do Hugging Face. Hoje ele ganha **cara de produto**: uma interface própria, bonita e responsiva, servida por uma API num serviço de produção, com endereço próprio e HTTPS. E a base de conhecimento passa a aceitar **PDF**.

O que não muda é o mais importante: o portão. O código continua sendo testado e a busca continua sendo avaliada antes de qualquer coisa ir para o ar. Muda o destino, não o fluxo.



GitHub Actions: o portão continua decidindo o que vai para o ar



Item	Parte 2	Parte 3
Interface	Gradio, padrão	React + Tailwind, própria, com cartões de fonte, tema escuro e celular
Servidor	app.py dentro do Space	FastAPI no Render: API + front-end no mesmo endereço
Endereço	huggingface.co/spaces/...	<servico>.onrender.com, com HTTPS
Documentos	Só .md	.md e .pdf, com conversão automática
Deploy	Actions envia ao Space	Actions dispara o Render pelo Deploy Hook e confere o site no ar
O que trazer	Supabase e documentos	Conta no Render e um PDF com texto da sua área

02 Do protótipo à produção

"Produção" não é um lugar, é um conjunto de garantias. Um protótipo prova que a ideia funciona; um sistema em produção precisa funcionar para os outros, todo dia, sem você olhando. As garantias que importam aqui:

Garantia	Pergunta que responde	Como resolvemos
Disponibilidade	Está no ar quando alguém acessa?	Serviço sempre ligado (plano pago) ou aviso de que o gratuito dorme
Reprodutibilidade	O que roda no servidor é o que eu testei?	Contêiner Docker: a mesma imagem no computador e no Render

Garantia	Pergunta que responde	Como resolvemos
Controle do deploy	Quem decide o que vai para o ar?	O portão no GitHub Actions; o Render não publica sozinho
Verificação	Como sei que o deploy deu certo?	Health check e um job que confere a versão no ar
Segurança	As chaves estão protegidas? Dá para abusar?	Chaves só no servidor; limites de uso
Observabilidade	Quando quebrar, como descubro o porquê?	Logs do Render e mensagens de erro claras
Reversão	Se a versão nova estiver ruim, como volto?	Rollback com um clique no Render; o índice anterior continua no Supabase

Por que sair do Hugging Face agora

O Space foi a escolha certa para aprender: grátis, com GPU sob demanda e o Gradio pronto. Para um produto, três limites pesam: a interface fica presa ao Gradio, o push recusa binários (por isso PDF não entrava) e a URL é a do Hugging Face. O Render é uma plataforma de aplicações (PaaS) mais próxima do que se usa em empresa: roda qualquer contêiner, tem domínio, health check, logs e rollback.

Plataforma	Bom para	Cuidados
Hugging Face Spaces	Demos de IA, Gradio, GPU	Interface e URL presas; binários recusados
Render	APIs e sites com contêiner, deploy por Git ou hook	Free dorme e tem 512 MB de RAM
Railway, Fly.io	Contêineres com regiões e escala	Modelo de preço por uso
Vercel, Netlify	Front-ends estáticos e funções curtas	Pouco adequados para servidor com modelo carregado
Azure App Service, Cloud Run, AWS	Produção corporativa	Mais configuração (rede, identidade, custos)

03 Front-end, back-end e a API entre eles

Até aqui, o Gradio escondia uma divisão que agora fica explícita. O **front-end** é o que roda no navegador do visitante: HTML, CSS e JavaScript. O **back-end** é o que roda no servidor: a busca, as chaves, a chamada ao modelo. Os dois conversam por uma **API**: um conjunto de endereços (rotas) que o servidor responde.

A regra que organiza tudo

Tudo que vai para o navegador é público: qualquer visitante pode abrir as ferramentas do navegador e ler o código. Por isso a interface nunca guarda chave, prompt de sistema nem nome de modelo. Ela manda a pergunta e recebe a resposta. É a mesma lição da Parte 1 sobre o GitHub Pages, agora aplicada ao nosso próprio front-end.

HTTP em uma tabela

Conceito	O que é	No projeto
Método	O tipo do pedido: GET lê, POST envia dados	GET /api/config; POST /api/perguntar
Rota	O endereço dentro do servidor	/api/saude, /api/config, /api/perguntar
Corpo (JSON)	Os dados enviados ou recebidos, em texto estruturado	{"mensagem": "...", "historico": [...]}
Código de status	O resultado: 2xx deu certo, 4xx erro de quem pediu, 5xx erro do servidor	200, 413 (grande demais), 422 (inválido), 429 (muitas perguntas), 503 (indisponível)
Cabeçalho	Informações extras do pedido ou da resposta	Content-Type, Retry-After, X-Forwarded-For

O contrato da API

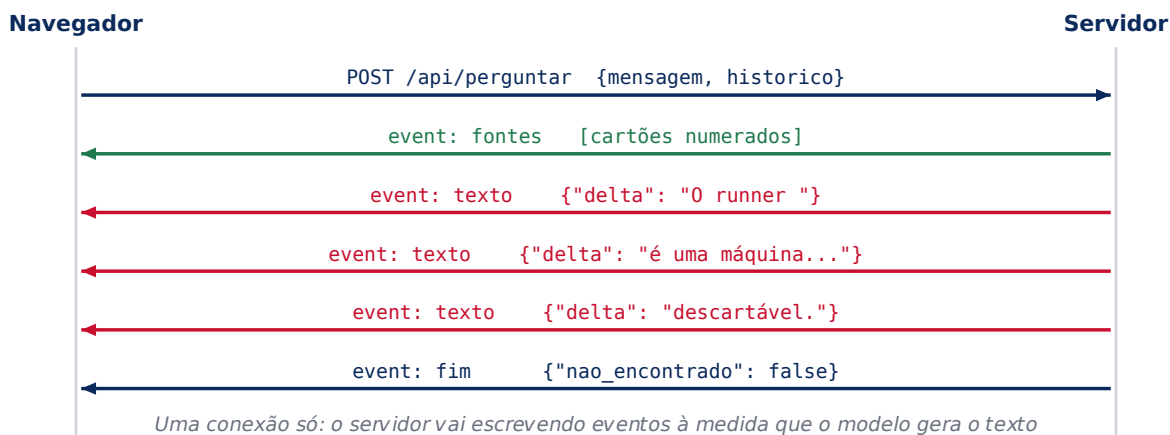
A spec define cada rota antes do código: o que recebe, o que devolve e quais erros. Esse contrato permite que o front-end e o back-end sejam construídos e testados separadamente, e é o que os testes automáticos conferem.

```
GET /api/saude      → {"status": "ok", "versao": "<commit>", "trechos_na_producao": 136}
GET /api/config    → {nome, descricao, exemplos, logo_url, cores, interface}   (sem segredos)
POST /api/perguntar ← {"mensagem": "...", "historico": [...]}
                    → text/event-stream: fontes, texto, texto, ..., fim
```

04 Streaming: a resposta aos poucos

Um modelo de linguagem gera a resposta pedaço por pedaço. Esperar o texto inteiro para mostrar deixa o usuário olhando para uma tela parada por segundos. Streaming é entregar cada pedaço assim que ele sai. Há três jeitos comuns de fazer isso na web:

Técnica	Como funciona	Quando usar
Server-Sent Events (SSE)	Uma resposta HTTP que fica aberta e recebe eventos de texto, só do servidor para o navegador	Chat com IA: é o que as APIs de modelos usam. O nosso caso
WebSocket	Canal aberto nos dois sentidos	Jogos, colaboração em tempo real, chat entre pessoas
Polling	O navegador pergunta de tempos em tempos se há novidade	Atualizações lentas; desperdiça requisições



Repare na ordem: as **fontes** chegam primeiro, porque a busca acontece antes da geração. O front-end já pode preparar os cartões e só mostrá-los no fim, se a resposta não for "não encontrei".

05 Front-end moderno: React, Vite e Tailwind

React: a interface em componentes

No React, a tela é montada com **componentes**: funções que recebem dados e devolvem um pedaço de interface. Quando o **estado** muda (uma mensagem nova, um pedaço de texto que chegou), o React redesenha só o que mudou. A interface do projeto fica dividida assim:

Componente	O que mostra	Estado que usa
Cabecalho	Logo, nome, descrição, botão de tema	configuração vinda de /api/config
Sugestoes	As perguntas de exemplo na tela inicial	se a conversa está vazia
Conversa e Mensagem	Os balões, com Markdown e botão de copiar	lista de mensagens
CartaoFonte	fonte > seção; ao clicar, o trecho usado	aberto ou fechado
CaixaPergunta	Campo de texto, contador, botão de enviar	texto digitado, se está respondendo
AlternarTema	Claro ou escuro	preferência salva no navegador

Vite: do código ao site

O navegador não entende JSX nem Tailwind diretamente. O **Vite** faz duas coisas: em desenvolvimento, um servidor local que recarrega a tela a cada alteração (`npm run dev`); para publicar, um **build** (`npm run build`) que gera a pasta `dist/` com HTML, CSS e JavaScript prontos e compactados. É essa pasta que o servidor entrega.

```

cd frontend
npm install      # instala as dependências (package.json)
npm run dev      # http://localhost:5173, recarrega sozinho; /api vai para a porta 8000
npm run build    # gera frontend/dist, o que vai para produção
  
```

Tailwind: estilo sem escrever CSS do zero

Tailwind é uma biblioteca de classes utilitárias: em vez de criar um arquivo de estilo, você descreve o visual na própria marcação, como `rounded-2xl bg-white shadow-sm dark:bg-slate-900`. Ela facilita manter espaçamentos, cores e o modo escuro consistentes, e é um formato que os assistentes de IA geram bem.

06 Interface de chat que parece produto

A diferença entre "funciona" e "parece produto" está nos detalhes. Uma lista do que a spec pede e por quê:

Detalhe	Por que importa
Estado vazio com sugestões	Ninguém sabe o que perguntar diante de uma tela em branco
Indicador de "pensando"	Entre o envio e o primeiro pedaço há a busca e a fila do modelo; sem sinal, parece travado
Resposta em Markdown	Listas, negrito e código ficam legíveis, não um bloco de texto
Cartões de fonte expansíveis	Mostram de onde veio a resposta e deixam o usuário conferir: é a confiança do RAG visível
Erro com "tentar de novo"	Falhas acontecem (modelo lotado, rede); o usuário não pode perder a pergunta
Aviso no limite de uso	Dizer quanto tempo esperar é melhor que um erro genérico
Celular e modo escuro	Boa parte do acesso é pelo celular, e muita gente usa tema escuro
Acessibilidade	Contraste, teclado e rótulos: quem usa leitor de tela também precisa conseguir

Hierarquia visual em três camadas

- **Cores de marca** com moderação: a cor principal do `config.yml` no cabeçalho, no botão de enviar e nos detalhes; o resto em tons neutros.
- **Tipografia** com no máximo dois pesos e tamanhos consistentes: a resposta precisa ser confortável de ler em parágrafos longos.
- **Espaço**: margens generosas e largura máxima de leitura (cerca de 70 caracteres por linha) valem mais que qualquer efeito.

07 Gerando o front-end com IA

Assistentes de programação geram interfaces muito boas quando recebem um pedido preciso, e interfaces genéricas quando o pedido é vago. A spec cumpre esse papel: lista os componentes, os estados, as cores e o contrato da API. Para o visual, acrescente referências concretas:

Implemente a tarefa 3 da SPEC-parte3. Visual: interface de chat limpa e moderna, inspirada nos chats comerciais. Largura máxima de leitura de 768 px, centralizada. Cor principal vinda de `/api/config`. Cantos arredondados, sombras suaves, tons neutros (slate) no fundo, modo escuro com `dark:` do Tailwind. Mensagens do usuário alinhadas à direita na cor principal; do assistente à esquerda, sem balão, em Markdown. Cartões de fonte em grade abaixo da resposta. Mostre como testar.

O que revisar no que a IA gerou

- **Dependências:** cada pacote novo em package.json é código de terceiros no seu site. Pergunte para que serve cada um e prefira poucos.
- **Segredos:** procure `import.meta.env` e `VITE_`. Nada de chave ali (o T17 também confere).
- **Rotas:** o front-end deve chamar só caminhos relativos (`/api/...`), nunca a API do provedor de IA diretamente.
- **Estados:** teste de propósito o erro (desligue o servidor), o 429 (pergunte rápido) e o "não encontrei".

08 O servidor: FastAPI

FastAPI é um framework Python para APIs. Cada rota é uma função; os dados de entrada são descritos com **Pydantic**, que valida tudo antes de a função rodar (por isso uma pergunta vazia já volta com 422, sem código extra). O que o `app.py` fazia com o Gradio é dividido em duas partes: `chat.py`, com os provedores e a geração da resposta, e `servidor.py`, com as rotas.

```
class Pergunta(BaseModel):
    mensagem: str = Field(min_length=1)
    historico: list[Mensagem] = []

@app.post("/api/perguntar")
def perguntar(dados: Pergunta, request: Request):
    verificar_limites(dados.mensagem, ip_do_visitante(request)) # 413 ou 429
    return StreamingResponse(eventos(dados), media_type="text/event-stream")

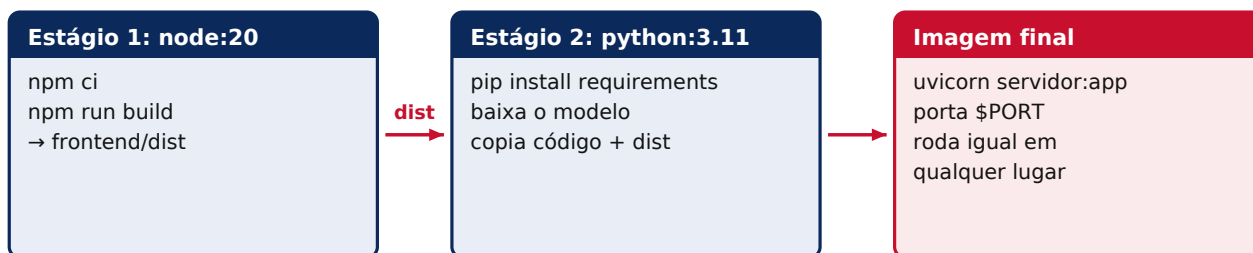
app.mount("/", StaticFiles(directory="frontend/dist", html=True)) # o front-end
```

A última linha faz o servidor entregar também o front-end compilado. Uma URL só para tudo: sem configuração de CORS (a regra do navegador que bloqueia chamadas entre endereços diferentes) e sem segundo serviço para manter.

09 Contêineres e Docker

A Parte 1 começou com a frase "na minha máquina funciona". O contêiner é a resposta mais completa a ela: em vez de mandar o código e torcer para o servidor ter as mesmas versões, você manda uma **imagem** com o sistema, o Python, as bibliotecas, o modelo de embedding e o código, tudo junto. A imagem roda igual no seu computador, no GitHub Actions e no Render.

Termo	O que é
Dockerfile	A receita da imagem: um passo por linha (base, instalar, copiar, iniciar)
Imagem	O resultado da receita: um pacote imutável e versionado
Contêiner	Uma imagem em execução
Multi-stage	Receita em estágios: um estágio compila e outro só recebe o resultado. A imagem final fica menor e sem as ferramentas de build



Por que baixar o modelo no build

Se o modelo de embedding fosse baixado a cada início do serviço, cada reinício levaria mais tempo e dependeria do Hugging Face estar no ar. Baixado no build, ele vem dentro da imagem: o serviço sobe mais rápido e sem dependência externa.

10 Render: o deploy de produção

No Render, cada aplicação é um **serviço**. O nosso é um **Web Service** com runtime Docker. Em vez de configurar tudo por cliques, descrevemos o serviço num arquivo do repositório, o `render.yaml`. Isso é **infraestrutura como código**: a configuração é versionada, revisada e reproduzível como o resto do projeto.

```

services:
- type: web
  name: professor-nta           # vira professor-nta.onrender.com
  runtime: docker
  plan: free
  healthCheckPath: /api/saude  # o Render só troca a versão se esta rota responder
  autoDeployTrigger: off      # quem publica é o GitHub Actions
  envVars:
    - key: OPENROUTER_API_KEY
      sync: false              # o valor é preenchido no painel, nunca aqui
  
```

Os recursos do Render que usamos

Recurso	O que faz	Por que importa
Blueprint	Cria o serviço a partir do <code>render.yaml</code>	Configuração reproduzível
Environment	Variáveis de ambiente cifradas do serviço	Onde ficam as chaves (o segredo fica com quem executa)
Health check	Chama <code>/api/saude</code> antes de trocar a versão	Versão que não sobe não substitui a que está no ar
Deploy Hook	Um endereço secreto que dispara um deploy	O GitHub Actions publica só depois do portão
Logs e Metrics	Saída do servidor, uso de CPU e memória	Diagnosticar erros e falta de memória
Rollback	Volta para um deploy anterior com um clique	Recuperação rápida

Planos e limites

Plano	Memória	Dorme?	Para quê
Free	512 MB	Sim, após cerca de 15 min sem acesso; leva perto de 1 min para acordar	Aprender e testar
Starter	512 MB	Não	Demonstração e uso leve sempre disponível
Standard	2 GB	Não	Se o modelo e o servidor não couberem em 512 MB

Limites e preços de planos mudam: confira no site do Render antes de escolher.

Por que desligar o Auto-Deploy

O Render pode publicar sozinho a cada commit, como a Parte 1 já tinha avisado. Ligado, ele ignora o portão: um commit com a busca piorada iria para o ar. Desligado, só o job publicar, depois do testar e do avaliar, chama o Deploy Hook.

11 URL, HTTPS e domínio

Todo Web Service do Render ganha um subdomínio `<nome>.onrender.com` com **HTTPS** automático: a conexão é cifrada por um certificado TLS que o Render emite e renova sozinho. O nome vem do campo `name` do `render.yaml`, se estiver livre; se não, o Render acrescenta um sufixo.

Para quando quiser um domínio próprio

1. Compre um domínio (registro.br para .com.br) ou use um que você já tem.
2. No Render: Settings → Custom Domains → Add, por exemplo `assistente.seudominio.com.br`.
3. No painel do seu domínio (DNS), crie um registro **CNAME** apontando para o `<nome>.onrender.com`, como o Render indicar.
4. O Render verifica o DNS e emite o certificado HTTPS sozinho.

12 PDF na base de conhecimento

Na Parte 2 a pasta `documentos/` aceitava só Markdown, por dois motivos: o Hugging Face recusava binários e o PDF não guarda estrutura. Saindo do Hugging Face, o primeiro motivo some. O segundo continua, e agora quem resolve é o pipeline: o `indexar.py` converte cada PDF para Markdown antes de dividir em trechos.

Problema do PDF	O que a conversão faz
Títulos não marcados	pymupdf4llm identifica títulos pelo tamanho da fonte e gera # e ##
Cabeçalho e rodapé em toda página	Linhas repetidas em mais da metade das páginas são removidas
Números de página soltos	Removidos
Tabelas	Convertidas em tabelas Markdown (as complexas podem sair imperfeitas)
PDF escaneado (só imagem)	Sem texto para extrair: a indexação para e avisa (OCR fica fora)

O portão protege a conversão

Conversão automática não garante qualidade. Por isso cada PDF novo deve ganhar perguntas em perguntas_teste.yml: se a extração vier ruim, a busca não acha o trecho, o hit rate cai e o deploy é barrado antes de chegar ao público.

Sobre licenças: o pymupdf4llm usa a licença AGPL. No projeto, ele roda só na indexação, no GitHub Actions, e não faz parte do servidor publicado. Uma alternativa com licença MIT é o MarkItDown, da Microsoft, que preserva menos a estrutura dos títulos.

13 Segurança e custo em produção

Risco	O que pode acontecer	Proteção no projeto
Chave no front-end	Qualquer visitante copia e usa a sua chave	Front-end só chama /api; T17 procura chaves no build; nada de VITE_ com segredo
Abuso da API	Alguém dispara milhares de perguntas e consome sua cota ou seu crédito	Limite de perguntas por minuto por visitante (429) e de tamanho (413)
Entrada maliciosa	Pedir para trocar o prompt ou o modelo	A API só aceita mensagem e histórico; prompt e modelo ficam no servidor
Injeção de prompt por documento	Um trecho com ordens para o modelo	Trechos delimitados e instrução para ignorar ordens dentro deles (Parte 2)
Vazamento em logs	Perguntas de usuários guardadas sem aviso	Logs só com tamanho, tempo e erro
Chave secreta do banco exposta	Alguém grava ou apaga a base	Servidor usa só a publicável; a secreta fica no GitHub

Por que o limite fica no servidor

Um limite feito só no front-end (desabilitar o botão, por exemplo) não protege nada: quem quiser abusar chama a API direto, sem passar pela interface. Toda regra de segurança precisa valer no servidor.

14 Observabilidade: saber o que está acontecendo

- **Health check** (/api/saude): diz se o servidor está pronto, qual versão está no ar e quantos trechos há na produção. O Render usa para decidir se troca a versão.
- **Verificação pós-deploy**: o job verificar consulta o health check até a versão no ar ser o commit que acabou de passar no portão. Se o Render não subir, o workflow fica vermelho e você fica sabendo.
- **Logs**: o painel do Render mostra a saída do servidor em tempo real. Mensagens claras (como as de diagnóstico da Parte 2) economizam horas.
- **Métricas**: uso de memória e CPU. Com 512 MB, acompanhe a memória no primeiro deploy.

```
# job verificar (ideia): espera a versão nova responder
for i in $(seq 1 30); do
  versao=$(curl -fsS "$RENDER_URL/api/saude" | python -c "import sys,json;print(json.load(sys.stdin)['versao'])")
  [ "$versao" = "$GITHUB_SHA" ] && echo "Versão nova no ar." && exit 0
  sleep 20
done
echo "A versão nova não subiu em 10 minutos."; exit 1
```

15 O projeto de referência por dentro

```
agente-teste/
├── frontend/
│   ├── src/components/
│   └── vite.config.js
├── servidor.py
├── chat.py
├── rag.py
├── indexar.py, avaliar.py
├── documentos/
├── testes.py, testes_api.py
├── Dockerfile
├── render.yaml
└── .github/workflows/deploy.yml
```

React + Vite + Tailwind
 Cabecalho, Conversa, Mensagem, CartaoFonte, ...
 proxy de /api para a porta 8000 em desenvolvimento
 FastAPI: /api/saude, /api/config, /api/perguntar + front-end
 provedores de IA e geração da resposta (vem do app.py)
 agora lê .md e .pdf
 como na Parte 2
 .md e .pdf
 T1 a T17
 imagem em dois estágios
 o serviço do Render como código
 testar → avaliar → publicar → verificar

Chave ou variável	Onde fica	Quem usa
SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, chave do provedor	Environment do Render	servidor.py
SUPABASE_URL, SUPABASE_SECRET_KEY	Secrets do GitHub	avaliar e publicar
RENDER_DEPLOY_HOOK_URL	Secret do GitHub	publicar
RENDER_URL (não é segredo)	Variável do GitHub	verificar

O HF_TOKEN e o Space deixam de ser usados. O Space pode ser pausado ou apagado quando o Render estiver no ar.

16 Da ideia à spec da Parte 3

O processo é o mesmo das partes anteriores: o IDEIA-parte3.md em linguagem simples (Apêndice A), o prompt que está no fim dele para gerar a SPEC-parte3-frontend-producao.md, a revisão e as tarefas uma por vez. Na revisão, confira:

- Alguma chave aparece no front-end, no render.yaml ou em variável VITE_?
- O Auto-Deploy do Render está desligado e quem publica é o job do GitHub Actions?
- Os limites de uso valem no servidor, não só na interface?
- O contrato da API lista cada rota, os dados e os erros?
- Os estados da interface (vazio, carregando, erro, não encontrei, limite) estão descritos?
- PDF escaneado tem comportamento definido?
- Os testes das Partes 1 e 2 continuam valendo, com os ajustes explícitos (T6 e T11)?

17 Passo a passo da aula

A. Preparar o Render

1. Crie a conta em render.com, entrando com o GitHub.
2. Autorize o Render a ler o seu repositório.

B. Da ideia ao código

1. Crie o IDEIA-parte3.md a partir do Apêndice A e gere a spec com o prompt dele.
2. Revise pela lista da seção 16 e aprove.
3. Implemente uma tarefa por vez. Para o front-end, use o pedido de visual da seção 07 e itere até ficar com a sua cara.

C. Publicar

1. No Render: New → Blueprint, escolha o repositório e preencha as variáveis pedidas.
2. Em Settings do serviço, copie o Deploy Hook e cadastre no GitHub como RENDER_DEPLOY_HOOK_URL. Cadastre também a variável RENDER_URL com o endereço do serviço.
3. Faça um commit. Acompanhe os quatro jobs na aba Actions e o deploy no painel do Render.

D. Testar

1. Abra o endereço no computador e no celular; alterne o tema.
2. Faça uma pergunta do material e abra os cartões de fonte.
3. Coloque um PDF com texto em documentos/, acrescente uma pergunta de teste sobre ele e faça commit.
4. Abra o código-fonte da página e procure qualquer coisa parecida com uma chave.

18 Erros comuns

Sintoma	Causa provável	Conserto
Serviço reinicia, log com falta de memória	512 MB não bastam	Conferir se o modelo é carregado uma vez; se persistir, plano Standard
Primeiro acesso demora cerca de 1 minuto	Plano Free dormindo	Normal; abrir antes da aula ou Starter
Render publicou sem passar no portão	Auto-Deploy ligado	Desligar no render.yaml e no painel
Job verificar estoura o tempo	Build lento, serviço não subiu ou RENDER_URL errada	Log de deploy no Render; conferir a variável
Página em branco	dist ausente na imagem ou base errada no Vite	Conferir o estágio de build e base: "/"
Chat com erro de rede no computador	API local fora do ar ou proxy do Vite errado	Subir o uvicorn na porta 8000
Chave no JavaScript publicado	Variável VITE_ com segredo	Remover, revogar a chave, manter só no servidor
PDF sem texto extraível	PDF escaneado	Usar um PDF com texto ou fazer OCR fora do projeto
Muitos 429 numa sala de aula	Todos atrás do mesmo IP da rede	Aumentar perguntas_por_minuto

19 Apêndice A: modelo de ideia da Parte 3

Resumo do IDEIA-parte3.md do projeto de referência. O arquivo completo, com o prompt para gerar a spec, está no repositório.

```
# Ideia do projeto · Parte 3: front-end próprio e deploy em produção

## O que eu quero
Dar ao assistente cara de produto: interface própria, bonita e moderna, num endereço próprio, com deploy mais robusto. E a base de conhecimento passa a aceitar PDF.

## Como eu quero que a interface seja
- Bonita e moderna, com as cores e a logo do config.yml; resposta aos poucos.
- Fontes como cartões clicáveis que mostram o trecho usado.
- Sugestões na tela inicial; celular; modo claro e escuro; tudo em português.

## Como eu quero que funcione por trás
- O navegador nunca recebe chave; um único endereço serve interface e API.
- Limites de tamanho e de perguntas por minuto, para ninguém gastar a minha chave.

## Documentos em PDF
- .pdf na pasta documentos/, convertido para Markdown na indexação.
- PDF escaneado: o portão avisa com mensagem clara.

## Onde eu quero publicar
- No Render (professor-nta.onrender.com), com HTTPS; configuração num arquivo do repositório.
- O GitHub Actions testa, avalia a busca e só então manda o Render publicar; depois confere o site.

## O que NÃO entra agora
- Login, conversas salvas, envio de documentos pela interface, domínio próprio, OCR.
```

20 Glossário

API: conjunto de rotas que um servidor responde, com um contrato de entrada e saída.

Back-end: a parte que roda no servidor: busca, chaves, chamada ao modelo.

Blueprint: no Render, a criação de serviços a partir do render.yaml.

Build: a etapa que transforma o código-fonte no que vai para produção (no Vite, a pasta dist).

CNAME: registro de DNS que aponta um nome para outro; usado no domínio próprio.

Componente: no React, uma função que devolve um pedaço da interface.

Contêiner: uma imagem Docker em execução.

CORS: regra do navegador para chamadas entre endereços diferentes; evitada com uma URL só.

Deploy Hook: endereço secreto que dispara um deploy no Render.

Dockerfile: a receita de uma imagem Docker.

FastAPI: framework Python para APIs, com validação pelo Pydantic.

Front-end: a parte que roda no navegador: HTML, CSS e JavaScript.

Health check: rota que diz se o serviço está pronto; usada pelo Render e pelo job verificar.

HTTPS / TLS: conexão cifrada por certificado; automática no Render.

Imagem: pacote imutável com sistema, bibliotecas e código, gerado pelo Dockerfile.

Infraestrutura como código: configuração de servidores descrita em arquivos versionados.

Multi-stage: Dockerfile em estágios: um compila, outro só recebe o resultado.

PaaS: plataforma que executa sua aplicação cuidando da infraestrutura.

Pydantic: biblioteca que valida dados de entrada pelo tipo declarado.

Rate limit: limite de pedidos por tempo; protege custo e disponibilidade.

React: biblioteca JavaScript para interfaces em componentes.

Rollback: voltar para uma versão anterior do deploy.

Server-Sent Events: resposta HTTP aberta pela qual o servidor envia eventos ao navegador.

Status HTTP: código que resume o resultado de um pedido (200, 413, 422, 429, 503).

Tailwind CSS: biblioteca de classes utilitárias de estilo.

Vite: ferramenta de desenvolvimento e build de front-end.